## Why do you need to test Django apps?

Testing your app is necessary before you go for launch. Users don't like half-baked products. So, investing in developing a web app and leaving your product's fate on pure luck does not work. Even if you are an upcoming developer and want to make the portfolio healthy, you need to ace the testing process.

Let's take the example of a library class. Here, an Author model is used. The best part about testing the Author model here is the redundancy of defining the character field in a database. So, you don't necessarily try first_name and last_name exclusively. Similarly, Django tests the date_of_birth field automatically for you, making the test more proficient.

Here is an example of the Author model:

```
class Author(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=150)
    date_of_birth = models.DateField(null=True, blank=False)
    date_of_death = models.DateField('Died', null=True, blank=False)

    def get_absolute_url(self):
        return reverse('author-detail', args=[str(self.id)])

    def __str__(self):
        return '%s, %s' % (self.last_name, self.first_name)
```

Now that we understand the need for testing, let's understand automated testing for Django apps.

Similarly, you should check that the custom methods behave as required because they are your code/business logic. In the case of get_absolute_url(), you can trust that the Django reverse() method has been implemented correctly, so what you're testing is whether the associated view has been defined.

Developers can leverage custom methods such as get_absolute_url() and _str_ () to test your web applications' business logic. Django reverse() practice covers the business logic part well with proper implementation. Developers must constrain the date of birth and death values to set up a